

CSS Functions (Функции CSS)

Введение

Функции CSS позволяют выполнять вычисления и манипуляции со значениями свойств. Они делают стили более динамичными и гибкими, позволяя создавать сложные эффекты и адаптивные макеты без использования JavaScript.

Математические функции

calc()

Позволяет выполнять математические вычисления с разными единицами измерения.

```
.container {  
  width: calc(100% - 40px);  
  padding: calc(1rem + 5px);  
  margin-top: calc(var(--spacing) * 2);  
}
```

min()

Возвращает наименьшее из перечисленных значений.

```
.responsive-width {  
  width: min(90%, 1200px); /* Будет использовать 90% ширины, но не более 1200px */  
}
```

max()

Возвращает наибольшее из перечисленных значений.

```
.text {  
  font-size: max(16px, 1.2vw); /* Не меньше 16px, но может быть больше в зависимости от ширины экрана */  
}
```

clamp()

Удерживает значение в заданном диапазоне.

```
.responsive-text {
  font-size: clamp(16px, 4vw, 32px); /* Минимум 16px, максимум 32px, между ними
- 4vw */
}

.container {
  width: clamp(320px, 80%, 1200px); /* Адаптивная ширина с ограничениями */
}
```

Функции для работы с цветом

rgb() и rgba()

Определяют цвет по значениям красного, зеленого и синего (и прозрачности для rgba).

```
.element {
  color: rgb(255, 0, 0); /* Красный */
  background-color: rgba(0, 0, 255, 0.5); /* Полупрозрачный синий */
}
```

hsl() и hsla()

Определяют цвет по тону, насыщенности и светлоте (и прозрачности для hsla).

```
.element {
  color: hsl(0, 100%, 50%); /* Красный */
  background-color: hsla(240, 100%, 50%, 0.5); /* Полупрозрачный синий */
}
```

hwb()

Определяет цвет по тону, белизне и черноте.

```
.element {
  color: hwb(0 0% 0%); /* Красный */
  background-color: hwb(240 0% 0% / 0.5); /* Полупрозрачный синий */
}
```

color-mix()

Смешивает два цвета в заданной цветовой модели.

```
.mixed-color {  
  background-color: color-mix(in srgb, #ff0000 50%, #0000ff 50%);  
}
```

color-contrast()

Выбирает цвет с наилучшим контрастом из списка относительно базового цвета.

```
.text {  
  color: color-contrast(var(--background-color) vs black, white, #777);  
}
```

Функции для градиентов

linear-gradient()

Создает линейный градиент.

```
.gradient {  
  background: linear-gradient(to right, red, orange, yellow, green, blue,  
    indigo, violet);  
  background: linear-gradient(45deg, rgba(0,0,255,1) 0%, rgba(0,255,0,0.5) 50%,  
    rgba(255,0,0,0) 100%);  
}
```

radial-gradient()

Создает радиальный градиент.

```
.radial {  
  background: radial-gradient(circle, yellow, green);  
  background: radial-gradient(ellipse at top left, red, transparent);  
}
```

conic-gradient()

Создает конический градиент.

```
.conic {  
  background: conic-gradient(red, orange, yellow, green, blue, indigo, violet,  
    red);  
}
```

```
background: conic-gradient(from 45deg at 50% 50%, blue, red);  
}
```

Функции для трансформаций

translate(), translateX(), translateY(), translateZ(), translate3d()

Перемещают элемент.

```
.element {  
  transform: translate(50px, 20px);  
  transform: translateX(50px);  
  transform: translate3d(50px, 20px, 10px);  
}
```

scale(), scaleX(), scaleY(), scaleZ(), scale3d()

Изменяют размер элемента.

```
.element {  
  transform: scale(1.5);  
  transform: scaleX(1.2);  
  transform: scale3d(1.2, 1.5, 1);  
}
```

rotate(), rotateX(), rotateY(), rotateZ(), rotate3d()

Вращают элемент.

```
.element {  
  transform: rotate(45deg);  
  transform: rotateX(45deg);  
  transform: rotate3d(1, 1, 1, 45deg);  
}
```

skew(), skewX(), skewY()

Наклоняют элемент.

```
.element {  
  transform: skew(10deg, 20deg);  
  transform: skewX(10deg);  
}
```

matrix(), matrix3d()

Применяют матричное преобразование.

```
.element {  
  transform: matrix(1, 0.2, 0.2, 1, 10, 10);  
}
```

Функции для фильтров

blur()

Размывает элемент.

```
.blurred {  
  filter: blur(5px);  
}
```

brightness()

Изменяет яркость.

```
.bright {  
  filter: brightness(150%);  
}
```

contrast()

Изменяет контрастность.

```
.high-contrast {  
  filter: contrast(200%);  
}
```

grayscale()

Преобразует в оттенки серого.

```
.gray {  
  filter: grayscale(100%);  
}
```

hue-rotate()

Изменяет цветовой тон.

```
.hue-shifted {  
  filter: hue-rotate(90deg);  
}
```

invert()

Инвертирует цвета.

```
.inverted {  
  filter: invert(100%);  
}
```

opacity()

Изменяет прозрачность.

```
.semi-transparent {  
  filter: opacity(50%);  
}
```

saturate()

Изменяет насыщенность.

```
.saturated {  
  filter: saturate(200%);  
}
```

sepia()

Применяет эффект сепии.

```
.old-photo {  
  filter: sepia(90%);  
}
```

drop-shadow()

Добавляет тень.

```
.shadowed {  
  filter: drop-shadow(3px 3px 5px rgba(0, 0, 0, 0.5));  
}
```

Функции для генерации контента

attr()

Получает значение атрибута HTML-элемента.

```
a::after {  
  content: " (" attr(href) )";  
}
```

url()

Загружает ресурс по URL.

```
.background {  
  background-image: url('image.jpg');  
}
```

Практические примеры

Адаптивная типографика

```
body {  
  font-size: clamp(16px, calc(1rem + 0.5vw), 24px);  
  line-height: calc(1.1em + 0.5vw);  
}
```

Динамические отступы

```
.container {  
  padding: min(5vw, 50px);  
  margin: max(20px, 2vh) auto;  
}
```

Интерактивные эффекты

```
.card {  
  transition: transform 0.3s ease;  
}  
  
.card:hover {  
  transform: translate(0, -10px) scale(1.02) rotate(1deg);  
  filter: brightness(105%) drop-shadow(0 10px 15px rgba(0, 0, 0, 0.1));  
}
```

Темная тема с функциями цвета

```
:root {  
  --base-hue: 210;  
  --base-saturation: 50%;  
  
  --light-bg: hsl(var(--base-hue), 20%, 95%);  
  --light-text: hsl(var(--base-hue), var(--base-saturation), 20%);  
  
  --dark-bg: hsl(var(--base-hue), 20%, 15%);  
  --dark-text: hsl(var(--base-hue), var(--base-saturation), 90%);  
}  
  
body {  
  background-color: var(--light-bg);  
  color: var(--light-text);  
}  
  
@media (prefers-color-scheme: dark) {  
  body {  
    background-color: var(--dark-bg);  
    color: var(--dark-text);  
  }  
}
```